

OPERATING SYSTEMS

Course Code: **23CS202** • Semester: **2-2** • Department of Computer Science & Engineering

Course Scope & Objectives: Core operating system abstractions: process lifecycle, CPU scheduling, synchronization primitives, deadlocks, virtual memory management, and file systems.

Unit 1: Operating System Architecture & Process Management

Key Syllabus Topics: OS Roles, System Calls (fork, exec, wait), Monolithic vs Microkernel, Process Control Block (PCB), Process States, Context Switching, Inter-Process Communication (Pipes, Shared Memory, Message Queues), Multithreading Models.

Theoretical Framework & Concepts:

The kernel manages hardware abstraction and hardware privilege levels (User Mode vs Kernel Mode). A Process is a program in execution containing code, stack, heap, and PCB registers.

Implementation / Key Formula Reference:

```
pid_t pid = fork();
if (pid == 0) {
    // Child Process
} else if (pid > 0) {
    wait(NULL); // Parent Process
}
```

Frequently Asked University Exam Questions:

• **Q1: Explain the components of a Process Control Block (PCB).**

Solution: A: Process ID (PID), Process State, Program Counter (PC), CPU Registers, CPU Scheduling Info, Memory Management Limits, and Open File Descriptors.

Unit 2: CPU Scheduling Algorithms

Key Syllabus Topics: Scheduling Criteria (Throughput, Turnaround, Waiting, Response Times), First-Come First-Served (FCFS), Shortest Job First (SJF / SRTF), Round Robin (RR), Priority Scheduling, Multi-Level Feedback Queue (MLFQ).

Theoretical Framework & Concepts:

SJF is provably optimal for minimizing average waiting time but requires future burst predictions. Round Robin guarantees fairness and bounded response times using fixed time quantum slices.

Implementation / Key Formula Reference:

```
// Average Waiting Time = (Sum of all Waiting Times) / N
// Turnaround Time = Completion Time - Arrival Time
```

Frequently Asked University Exam Questions:

- **Q1: Explain the Convoy Effect in FCFS scheduling.**

Solution: A: When a large CPU-intensive process holds the processor, shorter I/O-bound processes queue behind it, drastically lowering overall CPU and device utilization.

Unit 3: Process Synchronization & Deadlocks

Key Syllabus Topics: Critical Section Problem, Mutual Exclusion, Peterson's Solution, Hardware Instructions (TestAndSet), Semaphores (Counting, Binary), Mutex Locks, Classical Problems (Producer-Consumer, Dining Philosophers, Readers-Writers), Deadlock Characterization (Coffman Conditions), Banker's Algorithm.

Theoretical Framework & Concepts:

The Critical Section requires Mutual Exclusion, Progress, and Bounded Waiting. Deadlocks require 4 simultaneous conditions: Mutual Exclusion, Hold & Wait, No Preemption, and Circular Wait.

Implementation / Key Formula Reference:

```
sem_wait(&mutex);
// Critical Section
sem_post(&mutex);
```

Frequently Asked University Exam Questions:

- **Q1: State and explain Banker's Algorithm for deadlock avoidance.**

Solution: A: It evaluates resource allocation requests against available matrices, granting resources only if the resulting system state remains in a Safe State where all processes can finish.

Unit 4: Memory Management & Virtual Memory

Key Syllabus Topics: Contiguous Allocation, Paging, Page Table Structure, Translation Lookaside Buffer (TLB), Segmentation, Virtual Memory, Demand Paging, Page Faults, Page Replacement Algorithms (FIFO, Optimal, LRU), Thrashing & Working Set Model.

Theoretical Framework & Concepts:

Paging eliminates external fragmentation by partitioning virtual memory into fixed Pages and physical memory into Frames. The TLB caches address translations for single-cycle memory access.

Implementation / Key Formula Reference:

```
// Logical Address: [ Page Number (p) | Offset (d) ]
// Physical Address: [ Frame Number (f) | Offset (d) ]
```

Frequently Asked University Exam Questions:

- **Q1: Explain Thrashing and how the Working Set Model resolves it.**

Solution: A: Thrashing occurs when CPU spends more time swapping pages than executing code. The Working Set Model monitors active pages per process, suspending processes if total memory exceeds physical capacity.

Unit 5: File Systems & Disk Scheduling

Key Syllabus Topics: File Attributes & Operations, Directory Structures, File Allocation Methods (Contiguous, Linked, Indexed / Inode), Free Space Management (Bitmaps), Disk Structure, Disk Scheduling (FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK), RAID Levels.

Theoretical Framework & Concepts:

Inodes in UNIX contain file metadata and direct/indirect block pointers. Disk scheduling algorithms minimize mechanical seek head movement across magnetic platters.

Implementation / Key Formula Reference:

```
// C-SCAN services requests in one direction, then jumps back to the beginning without servicing returns
```

Frequently Asked University Exam Questions:

- **Q1: Compare SCAN and C-SCAN disk scheduling algorithms.**

Solution: A: SCAN travels edge-to-edge servicing requests in both directions. C-SCAN services requests in only one direction and returns immediately to the start, providing more uniform waiting times.